



评测报告

MOSS-VL 模型 OCR 能力评测报告

华东理工大学 余喆浩

—— 二〇二六年七月 ——

摘 要

本报告对 MOSS-VL-0708 视觉语言大模型进行了系统性的 OCR 能力评测，并与 15 个代表性模型展开横向对比，覆盖专用 OCR 工具、开源 VLM、闭源 API 型 VLM、闭源商业大模型四类模型。评测基准包含 10 个数据集：CORD（收据键值提取）、FUNSD（表单理解）、OCRBench v1/v2（纯文字识别与复杂场景 VQA）、ChartQA（图表问答）、OmniDocBench（文档整页解析）、OCR-Reasoning（图表数值推理）、DocVQA（文档图像问答）、TextVQA（场景文字问答）和 InfoVQA（信息图表问答）。OCRBench v1 使用字符串相似度（NED）评分，其余所有任务使用 LLM-as-Judge（qwen3-vl-flash）评分。

评测结果表明，MOSS-VL-0708 在 OCRBench v1 纯文字识别任务上以 98.3% 的准确率位列第三，仅次于 Qwen3-VL-Plus（98.6%）与 HunyuanOCR-1.5（98.4%），超越所有参评闭源大模型（最高 96.4%）；在 FUNSD 表单关键信息提取任务上以 80.0% 位列第 3，优于全部闭源大模型；TextVQA 场景文字问答以 95.5% 位列第 1，超越全部参评模型；OCRBench v2 复杂场景 VQA 任务上得分 84.3%，位列第 9 名，明显优于专用 OCR 工具（最高 52.2%）。然而，MOSS-VL-0708 在复杂推理类任务上存在明显短板：OCR-Reasoning 仅得 27.5%（CoT 引导后），ChartQA（48.0%）和 InfoVQA（50.0%）亦较弱，在需要多步推理或复杂图表理解的任务上与顶级闭源模型差距明显。

综合分析表明，MOSS-VL-0708 作为 11B 参数量级的开源模型，在 OCR 文字识别与文档关键信息提取（KIE）领域具备接近或超越闭源大模型的竞争力，适合部署于对精度要求高的文字识别流水线；但在多步推理、复杂图表与信息图理解等需要深度语义理解的任务上，仍需进一步对齐训练。

关键词：视觉语言模型；多模态评测；OCR；文档理解；Benchmark

目 录

第 1 章 引 言.....	1
1.1 评测背景	1
1.2 评测模型概览	1
第 2 章 评测方案.....	3
2.1 数据集选取	3
2.2 评分方法	4
2.2.1 归一化编辑距离（NED）	4
2.2.2 LLM-as-Judge	4
2.2.3 评分细则	4
2.3 特殊 Prompt 处理	5
2.4 实验环境	5
第 3 章 评测结果与分析.....	6
3.1 完整对比表	6
3.2 纯文字识别类任务	7
3.2.1 OCRBench v1	7
3.2.2 OCRBench v2	8
3.2.3 关键信息提取（KIE）	9
3.3 文档理解类任务	10
3.3.1 OmniDocBench: 文档整页解析.....	10
3.4 推理与问答类任务	11
3.4.1 ChartQA: 图表问答	11
3.4.2 OCR-Reasoning: 图表数值推理	11
3.4.3 DocVQA / TextVQA / InfoVQA: 视觉问答.....	12
3.5 MOSS-VL-0708 逐任务综合分析	14
3.6 Qwen3-VL 规模扩展分析.....	15
第 4 章 评测结论.....	17
4.1 主要结论	17
4.2 MOSS-VL-0708 改进方向	18
4.2.1 多步推理能力能力	18
4.2.2 信息图表理解能力	18

4.2.3 文档定位读取能力	18
4.2.4 结构化文档提取能力	18
4.3 模型选型建议	19
4.4 评测局限性	19
参考文献.....	21
附录 A 补充材料	22

符号和缩略语说明

VLM	视觉语言大模型 (Vision-Language Model)
OCR	光学字符识别 (Optical Character Recognition)
KIE	关键信息提取 (Key Information Extraction)
VQA	视觉问答 (Visual Question Answering)
NED	归一化编辑距离 (Normalized Edit Distance)
LLM	大语言模型 (Large Language Model)
CoT	思维链 (Chain-of-Thought)
API	应用程序接口 (Application Programming Interface)
pp	百分点 (Percentage Point)
LLM Judge	基于大语言模型的自动评分方法 (LLM-as-Judge)

第 1 章 引 言

1.1 评测背景

近年来，以 GPT-4V、Gemini、Qwen-VL 为代表的视觉语言大模型在光学字符识别（Optical Character Recognition, OCR）、文档理解、图表问答等多模态任务上取得了显著进展，正逐步替代传统专用 OCR 工具，成为文档智能处理领域的核心技术路线^[1-2]。

然而，当前多模态大模型的评测体系仍不成熟：各研究团队往往仅在自有榜单上报告成绩，缺乏跨模型、跨任务维度的统一横向比较；评分方法的差异（字符串精确匹配 vs. LLM-as-Judge vs. 数值容忍匹配）导致不同论文的数字难以直接比较；开源模型与闭源商业模型、专用 OCR 工具之间的能力边界尚未得到系统性梳理^[3-4]。

MOSS-VL-0708 是 OpenMOSS 团队开发的 11B 参数量级开源视觉语言模型，本报告通过标准化的评测框架，客观评估 MOSS-VL-0708 在多个权威 Benchmark 上的实际能力，并与专用 OCR 工具、主流开源 VLM、顶级闭源商业大模型进行系统性横向比较。

1.2 评测模型概览

本次评测共涉及 15 个模型，按照技术路线分为五组：

（1）MOSS-VL 模型

MOSS-VL-0708 为本次评测的核心对象，参数量 11B，针对 OCR 和文档关键信息提取任务进行了专项训练。

（2）专用 OCR 模型

包括 PaddleOCR-VL-1.6、PP-OCRv5（均基于百度智能云 OCR API）、GLM-OCR 和 DeepSeek-OCR-2，这些模型为文档结构识别和文字提取专门设计，但通常不支持开放式 VQA 任务。

（3）开源视觉语言模型

Qwen3-VL 开源尺寸（8B / 32B / 235B）与 HunyuanOCR-1.5（1B），代表本次评测中的开源 VLM 路线^[5]。

（4）闭源视觉语言模型

Qwen3-VL-Flash、Qwen3-VL-Plus 与 Qwen-VL-OCR（阿里云）作为闭源 VLM

的对比参照。

（5）闭源商业大模型

GPT-5-Sol、Claude-Sonnet-5 和 Gemini-3.6-Flash，代表当前不同级别的商业闭源模型最强水平，作为性能上界参照。

表 1.1 汇总了各参评模型的基本信息。

表 1.1 参评模型概览

模型名称	类别	参数量	主要特点
MOSS-VL-0708	开源 VLM	11B	OCR/KIE 专项优化
PaddleOCR-VL-1.6	专用 OCR	0.9B	百度智能云，文档提取
PP-OCRv5	专用 OCR	5M	百度智能云，版面分析
GLM-OCR	专用 OCR	0.9B	清华智谱，文档解析
DeepSeek-OCR-2	专用 OCR	3B	深度求索，多任务
Qwen3-VL-8B	开源 VLM	8B	Qwen3 最小尺寸
Qwen3-VL-32B	开源 VLM	32B	Qwen3 中等尺寸
Qwen3-VL-235B	开源 VLM	235B	Qwen3 旗舰尺寸
Qwen3-VL-Flash	闭源 VLM	—	Qwen3 API 快速版本
Qwen3-VL-Plus	闭源 VLM	—	Qwen3 API 增强版本
Qwen-VL-OCR	闭源 VLM	—	阿里云闭源 VLM
HunyuanOCR-1.5	开源 VLM	1B	腾讯混元 OCR 模型
GPT-5-Sol	闭源大模型	—	OpenAI 旗舰模型
Claude-Sonnet-5	闭源大模型	—	Anthropic 中等规模模型
Gemini-3.6-Flash	闭源大模型	—	Google 轻量级模型

第 2 章 评测方案

2.1 数据集选取

本次评测选取了 10 个覆盖不同文档智能子任务的权威 Benchmark，涵盖纯文字识别、关键信息提取（KIE）、文档整页解析、图表问答与推理、视觉问答（VQA）五大类别。表 2.1 列出了各数据集的基本信息。

表 2.1 评测 Benchmark 汇总

数据集	任务类型	样本数	评分方法	主要参考
CORD	收据 KIE	50	LLM Judge	Park et al. ^[6]
FUNSD	表单 KIE	50	LLM Judge	Jaume et al. ^[7]
OCRBench v1	纯文字识别	100	NED 相似度	Liu et al. ^[3]
OCRBench v2	复杂场景 VQA	230	LLM Judge	Fu et al. ^[8]
ChartQA	图表问答	50	LLM Judge	Masry et al. ^[9]
OmniDocBench	文档整页解析	200	LLM Judge	Wang et al. ^[4]
OCR-Reasoning	图表数值推理	200	LLM Judge	Huang et al. ^[10]
DocVQA	文档图像 VQA	200	LLM Judge	Mathew et al. ^[11]
TextVQA	场景文字 VQA	200	LLM Judge	Singh et al. ^[12]
InfoVQA	信息图表 VQA	200	LLM Judge	Mathew et al. ^[13]

各数据集的具体说明如下：

CORD：收据关键值对提取数据集，要求模型从收据图像中准确提取商品名、数量、金额等结构化字段。本次随机抽取 50 个样本。

FUNSD：扫描版表单理解数据集，包含手写和印刷混合的表单图像，要求提取字段名称与对应值。本次随机抽取 50 个样本。

OCRBench v1：综合性 OCR 评测基准，覆盖场景文字、文档文字、数学公式等多种场景，使用归一化编辑距离（NED）评分，本次评测使用 100 个样本^[3]。

OCRBench v2：OCRBench 的升级版本，增加了更多复杂场景 VQA 样本，本次基于 230 个所有模型共同测试的样本进行对比，使用 LLM Judge 评分^[8]。

ChartQA：图表问答数据集，包含折线图、柱状图、饼图等多种类型，问题涉及数值读取、趋势判断和比较推理^[9]。

OmniDocBench：文档整页解析 Benchmark，包含四个子集：标准版（v1.5）、

公式难 (equation_hard)、版面难 (layout_hard)、表格难 (table_hard)，各 50 个样本^[4]。

OCR-Reasoning: 面向复杂文字密集图像的推理评测基准，要求模型读取图表数据并完成多步数值计算，期望输出包含完整推理过程^[10]。

DocVQA / TextVQA / InfoVQA: 三个视觉问答数据集，分别针对文档图像、场景图像中的文字、信息图表，均取 validation split 各 200 个样本^[11-13]。

2.2 评分方法

2.2.1 归一化编辑距离 (NED)

OCRBench v1 使用字符串相似度评分。对于每个样本，计算模型输出与参考文本的归一化编辑距离：

$$NED = 1 - \frac{\text{EditDistance}(\hat{y}, y)}{\max(|\hat{y}|, |y|)} \quad (2.1)$$

其中 $\hat{y}$ 为模型输出， y 为参考文本，评分忽略大小写差异。最终得分取所有样本 NED 的均值。

2.2.2 LLM-as-Judge

对于所有 VQA 类和 KIE 类任务，本次评测统一采用 LLM-as-Judge 方法，使用 qwen3-v1-flash 作为裁判模型。给定问题、参考答案和模型输出，裁判模型以 0/1 二值方式判断模型回答是否与参考答案语义等价，最终得分为正确样本比例。

该方法相比字符串精确匹配更宽容，能够处理同义表达、格式差异等情况；相比人工评判则具有更高的一致性和可扩展性。

2.2.3 评分细则

为保证跨模型比较的一致性，各任务的评分遵循以下细则：

- 全部 VQA 类与 KIE 类任务统一使用 LLM Judge (qwen3-v1-flash) 评分，采用相同的判分 prompt；
- TextVQA 以 10 个人工标注的众数投票作为参考答案；
- OCRBench v1 使用 NED 字符串相似度评分，忽略大小写差异；
- OCRBench v2 使用 LLM Judge 评分，基于 230 个所有模型共同测试的样本进行对比，确保公平性；
- 对于因图像尺寸超出 API 限制而无法处理的样本，得分基于成功返回的样本

计算（如 Claude-Sonnet-5 在 OmniDocBench 上完成 156/200 条）。

2.3 特殊 Prompt 处理

OCR-Reasoning 数据集要求输出完整的逐步推理过程，但 MOSS-VL-0708 在默认 prompt 下倾向于直接输出数值答案（无推理过程），导致 LLM Judge 无法验证推理链而判定为错误，初测得分仅 9.0%。

经分析，这是 prompt 对齐问题，而非模型能力问题。对此，针对 OCR-Reasoning 任务添加了 Chain-of-Thought (CoT) 引导 prompt，要求模型分步展示推理过程。添加 CoT 引导后，MOSS-VL-0708 得分从 9.0% 提升至 27.5%，提升幅度达 206%。

表 2.2 对比了 CoT prompt 对不同模型的影响。其他已具备自动 CoT 训练的模型（如 Qwen3-VL 系列）在相同简短 prompt 下即可自动展开推理，因此无需额外处理。

表 2.2 OCR-Reasoning 任务中 CoT Prompt 效果对比

模型	Prompt 类型	输出特点	得分
MOSS-VL-0708	简短问题（无 CoT）	直接输出数值，无推理	9.0%
MOSS-VL-0708	CoT 引导 prompt	分步推理后给出答案	27.5%
Qwen3-VL-Flash	简短问题（无 CoT）	自动展开推理	67.3%

报告中 MOSS-VL-0708 的 OCR-Reasoning 得分均使用 CoT 重测后的 27.5%，原始无 CoT 得分（9.0%）保留作为 baseline 参照。

2.4 实验环境

所有模型通过统一的 Python 评测框架 (eval/run_benchmark.py) 调用，使用相同的输入格式和输出解析逻辑。专用 OCR 模型通过百度智能云 API 调用；其余所有模型（开源及闭源商业模型）均通过代理 API 调用。评测结果存储于 results/scores/ 目录，原始输出和 LLM Judge 判分结果分别保存为 CSV 和 JSON 格式，便于复查和追溯。

第 3 章 评测结果与分析

3.1 完整对比表

表 3.1 给出完整的数值对比表。

表 3.1 完整 Benchmark 评测结果（%）

模型	CORD	FUNSD	OCRv1	OCRv2	ChartQA	OmniDoc	OCR-R	DocVQA	TextVQA	InfoVQA
MOSS-VL-0708	74.0	80.0	98.3	84.3	48.0	85.0	27.5	90.0	95.5	50.0
PaddleOCR-VL-1.6	85.7	78.0	91.5	31.7	—	73.6	—	—	—	—
PP-OCRv5	85.7	78.0	91.5	30.9	—	73.9	—	—	—	—
GLM-OCR	56.0	48.0	45.8	38.7	—	<u>94.4</u>	—	—	—	—
DeepSeek-OCR-2	72.0	54.0	76.3	52.2	32.0	70.7	—	51.0	58.0	20.3
Qwen3-VL-Plus	82.0	84.0	98.6	<u>91.3</u>	66.0	92.1	67.8	<u>98.5</u>	86.0	78.0
Qwen3-VL-235B	<u>88.0</u>	<u>82.0</u>	97.9	92.6	66.0	91.3	67.0	96.5	80.0	73.5
Qwen3-VL-Flash	80.0	76.0	97.3	88.7	68.0	93.0	67.3	97.5	47.0	69.5
Qwen3-VL-8B	82.0	<u>82.0</u>	98.2	87.4	52.0	87.6	48.5	89.5	60.0	64.0
Qwen3-VL-32B	82.0	76.0	97.1	92.2	66.0	88.7	64.5	97.5	69.0	77.0
GPT-5-Sol	84.0	63.2	96.1	<u>92.6</u>	<u>82.0</u>	89.3	66.8	98.0	79.5	<u>88.0</u>
Claude-Sonnet-5	80.0	59.0	96.4	91.7	78.0	95.5	<u>74.3</u>	97.5	86.5	89.5
Gemini-3.6-Flash	<u>88.0</u>	61.2	93.9	93.5	90.0	91.8	81.5	99.0	<u>90.9</u>	<u>88.0</u>
Qwen-VL-OCR	64.0	50.0	56.8	75.5	54.0	86.9	8.6	90.0	12.5	45.8
HunyuanOCR-1.5	98.0	78.0	<u>98.4</u>	81.2	56.0	72.5	11.5	91.0	86.0	54.5

注：粗体表示列内第一；下划线表示列内第二；“—”表示未测试。

OCR-R 表示 OCR-Reasoning；OCRv1/v2 表示 OCRBench v1/v2；OmniDoc 表示 OmniDocBench。

Claude-Sonnet-5 / OmniDocBench 仅 156/200 条成功；OCRBench v2 基于 230 个共同样本。

图 3.1 以雷达图对比 MOSS-VL-0708 与四个代表性模型（Qwen3-VL-Plus、Gemini-3.6-Flash、GPT-5-Sol、HunyuanOCR-1.5）在八个维度上的综合能力。

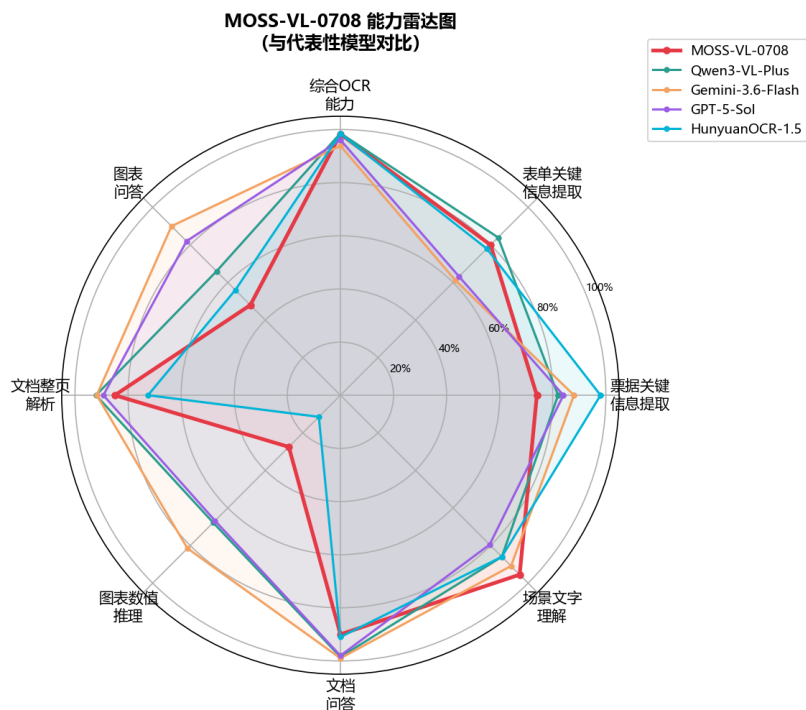


图 3.1 MOSS-VL-0708 能力雷达图（与代表性模型对比）

3.2 纯文字识别类任务

3.2.1 OCRBench v1

图 3.2 展示了各模型在 OCRBench v1 上的排名。

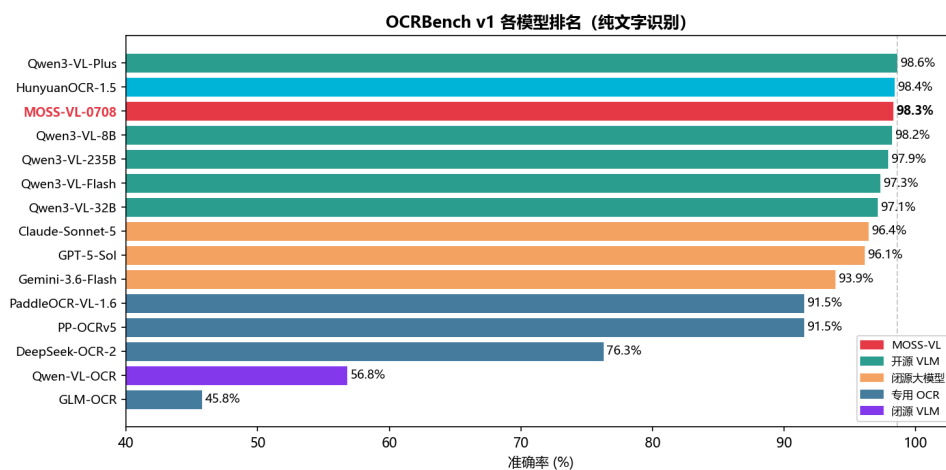


图 3.2 OCRBench v1 各模型排名（纯文字识别，NED 评分）

MOSS-VL-0708 以 98.3% 在 OCRBench v1 上位列第三，仅次于 Qwen3-VL-Plus (98.6%) 和 HunyuanOCR-1.5 (98.4%)，超越所有参评闭源大模型（最高 Claude-Sonnet-5 的 96.4%）和专用 OCR 工具（最高 91.5%）。

表 3.2 OCRBench v1 前 5 名

排名	模型	准确率
1	Qwen3-VL-Plus	98.6%
2	HunyuanOCR-1.5	98.4%
3	MOSS-VL-0708	98.3%
4	Qwen3-VL-8B	98.2%
5	Qwen3-VL-235B	97.9%
最强闭源（Claude-Sonnet-5）		96.4%
最强专用 OCR（PaddleOCR-VL-1.6）		91.5%

该结果表明，MOSS-VL-0708 在纯文字识别能力上已达到顶级开源 VLM 水平，并明显超越专用 OCR 工具。

3.2.2 OCRBench v2

OCRBench v2 整体难度显著高于 v1，且采用 LLM Judge 评分（而非 v1 的字符串相似度）。本次评测基于 230 个所有模型共同测试的样本进行横向比较，确保公平性。

MOSS-VL-0708 在 OCRBench v2 上得分 84.3%，位列第 9 名（共 15 个模型）。与顶级闭源大模型（Gemini 93.5%）相比差距为 9.2 个百分点，与最强开源模型 Qwen3-VL-235B（92.6%）差距为 8.3 个百分点；但相比专用 OCR 工具（最高 52.2%），MOSS-VL-0708 仍具备通用视觉语言模型在复杂场景 VQA 任务上的优势。

表 3.3 OCRBench v2 各模型得分（基于 230 个共同样本）

模型	得分	正确数/总数
Gemini-3.6-Flash	93.5%	215/230
Qwen3-VL-235B	92.6%	213/230
GPT-5-Sol	92.6%	213/230
Qwen3-VL-32B	92.2%	212/230
Claude-Sonnet-5	91.7%	211/230
Qwen3-VL-Plus	91.3%	210/230
Qwen3-VL-Flash	88.7%	204/230
Qwen3-VL-8B	87.4%	201/230
MOSS-VL-0708	84.3%	194/230
HunyuanOCR-1.5	81.2%	187/230
Qwen-VL-OCR	75.5%	173/229
DeepSeek-OCR-2	52.2%	120/230
GLM-OCR	38.7%	89/230
PaddleOCR-VL-1.6	31.7%	73/230
PP-OCRv5	30.9%	71/230

3.2.3 关键信息提取（KIE）

表 3.4 CORD 与 FUNSD 关键信息提取任务对比

模型	CORD	FUNSD
MOSS-VL-0708	74.0	80.0
PaddleOCR-VL-1.6 / PP-OCRv5	85.7	78.0
Qwen3-VL-Plus	82.0	84.0
Qwen3-VL-235B	88.0	82.0
Qwen3-VL-8B	82.0	82.0
HunyuanOCR-1.5	98.0	78.0
GPT-5-Sol	84.0	63.2
Claude-Sonnet-5	80.0	59.0
Gemini-3.6-Flash	88.0	61.2

在 FUNSD 表单 KIE 任务上，MOSS-VL-0708 以 80.0% 位列第 3（共 15 个模型），优于 GPT-5-Sol（63.2%）、Gemini-3.6-Flash（61.2%）、Claude-Sonnet-5（59.0%）等所有参评的商业闭源大模型，仅次于 Qwen3-VL-Plus（84.0%）和 Qwen3-VL-8B/235B（均为 82.0%）。这说明 MOSS-VL-0708 的 OCR 专项训练在结构化信息提取方面确有成效。

在 CORD 收据 KIE 任务上，MOSS-VL-0708 得分 74.0%，落后于 HunyuanOCR-1.5（98.0%）、专用 OCR 工具（85.7%）和多数开源 VLM，处于中游水平。

3.3 文档理解类任务

3.3.1 OmniDocBench：文档整页解析

OmniDocBench 是一个综合性的文档整页解析 Benchmark，包含四个子集：标准版（v1.5）、公式难（equation_hard）、版面难（layout_hard）、表格难（table_hard），各 50 个样本，共 200 个样本。该任务要求模型同时处理版面分析、表格还原、公式识别等复杂子任务。

表 3.5 OmniDocBench 各模型得分（文档整页解析）

模型	得分	备注
Claude-Sonnet-5	95.5%	仅 156/200 条成功
GLM-OCR	94.4%	
Qwen3-VL-Flash	93.0%	
Qwen3-VL-Plus	92.1%	
Gemini-3.6-Flash	91.8%	
Qwen3-VL-235B	91.3%	
GPT-5-Sol	89.3%	
Qwen3-VL-32B	88.7%	
Qwen3-VL-8B	87.6%	
Qwen-VL-OCR	86.9%	
MOSS-VL-0708	85.0%	
PP-OCrv5	73.9%	
PaddleOCR-VL-1.6	73.6%	
HunyuanOCR-1.5	72.5%	
DeepSeek-OCR-2	70.7%	

MOSS-VL-0708 在 OmniDocBench 上得分 85.0%，优于所有专用 OCR 工具（最高 73.9%），但落后于各通用 VLM 和闭源大模型（最低 86.9%）。在文档整页解析任务中，模型需要同时处理版面分析、表格还原、公式识别等复杂子任务，该结果表明 MOSS-VL-0708 在综合文档解析能力上仍有提升空间。

3.4 推理与问答类任务

3.4.1 ChartQA：图表问答

ChartQA 任务要求模型理解图表并回答涉及数值读取、趋势判断的问题。

表 3.6 ChartQA 各模型得分对比

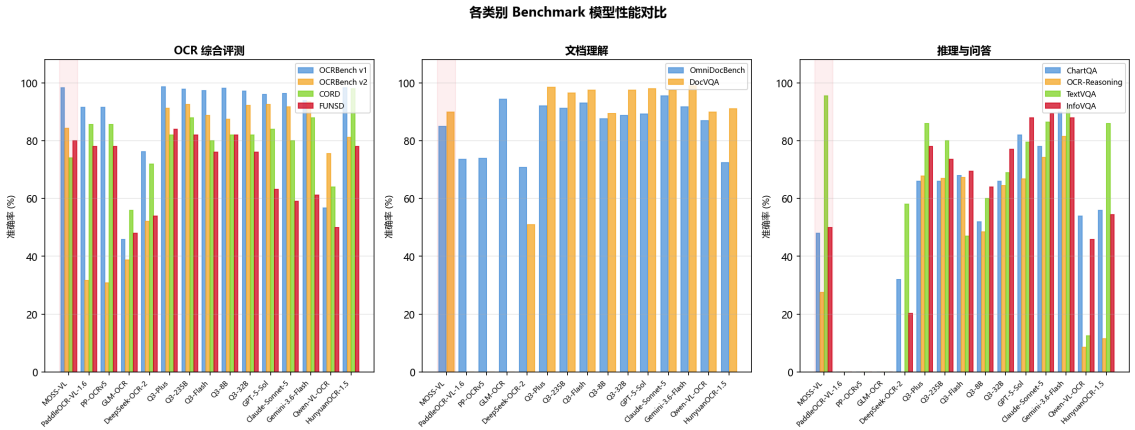
模型	得分
Gemini-3.6-Flash	90.0%
GPT-5-Sol	82.0%
Claude-Sonnet-5	78.0%
Qwen3-VL-Flash	68.0%
Qwen3-VL-Plus	66.0%
Qwen3-VL-235B	66.0%
Qwen3-VL-32B	66.0%
HunyuanOCR-1.5	56.0%
Qwen-VL-OCR	54.0%
Qwen3-VL-8B	52.0%
MOSS-VL-0708	48.0%
DeepSeek-OCR-2	32.0%

Gemini-3.6-Flash 以 **90.0%** 大幅领先，闭源大模型整体优势明显（78.0%–90.0%）；Qwen 系列 VLM（含 Flash/Plus API 版本及开源尺寸）主要处于 52.0%–68.0% 区间。MOSS-VL-0708 仅得 48.0%（第 9/12），与最强模型差距达 42 个百分点，反映出其在图表语义理解和推理能力上的明显不足。

3.4.2 OCR-Reasoning：图表数值推理

OCR-Reasoning 要求多步数值推理，是本次评测中难度最高的任务之一。如图 3.3 所示，Gemini-3.6-Flash 以 81.5% 夺冠，Claude-Sonnet-5 以 74.3% 居次，Qwen3-VL 系列集中在 48.5%–67.8% 区间。MOSS-VL-0708 使用 CoT 引导后得分 27.5%，

仍与最强模型相差约 54 个百分点，是本次评测中 MOSS-VL-0708 最突出的短板。



3.4.3 DocVQA / TextVQA / InfoVQA：视觉问答

表 3.7 VQA 类任务各模型得分对比（%）

模型	DocVQA	TextVQA	InfoVQA
MOSS-VL-0708	90.0	95.5	50.0
Qwen3-VL-Plus	<u>98.5</u>	86.0	78.0
Qwen3-VL-235B	96.5	80.0	73.5
Qwen3-VL-Flash	97.5	47.0	69.5
Qwen3-VL-8B	89.5	60.0	64.0
GPT-5-Sol	98.0	79.5	<u>88.0</u>
Claude-Sonnet-5	97.5	86.5	89.5
Gemini-3.6-Flash	99.0	<u>90.9</u>	<u>88.0</u>
HunyuanOCR-1.5	91.0	86.0	54.5
DeepSeek-OCR-2	51.0	58.0	20.3

所有 VQA 类任务均采用 LLM Judge（qwen3-vl-flash）评分，语义等价即判为正确。

TextVQA TextVQA 要求模型读取自然场景图像中的文字（商标、标牌等）并回答问题。MOSS-VL-0708 以 **95.5%** 位列所有参评模型第一，超越 Gemini-3.6-Flash（90.9%）、Claude-Sonnet-5（86.5%）等顶级闭源大模型，领先幅度为 4.6–16 个百分点，表明 MOSS-VL-0708 在场景文字识别方面已达到顶级水平。模型在典型样本

上表现准确，例如问题”what is the brand of this camera?”，模型输出”dakota digital”，标准答案”dakota”，语义正确。剩余问题主要来自两类情况：一是图像中模糊或细小的数字（如钟表上的”5:41”被误读为”10:10”）；二是被标注为”unanswerable”的样本，模型未能识别该类别而给出猜测性答案。

DocVQA DocVQA 要求模型阅读扫描文档并回答问题。MOSS-VL-0708 得分 90.0%，位列第 7，超越同参数量的 Qwen3-VL-8B（89.5%），但落后于顶级模型（Gemini 99.0%，Qwen3-VL-Plus 98.5%）约 9 个百分点。错误样例分析中能够发现两类主要问题：其一为数值读取偏差，例如问题”What is the ’actual’ value per 1000, during the year 1975?”，标准答案”0.28”，模型输出”0.24”，为图表刻度读取误差；其二为文档定位错误，对会议出席次数等表格数据，模型倾向于输出”[1]”而非正确值”2”，表现出文档表格理解的不稳定性。

InfoVQA InfoVQA 要求模型理解含有大量文字和数据关系的信息图表。MOSS-VL-0708 得分 50.0%，排名第 9（共 12 个模型），落后于 Qwen3-VL-8B（64.0%）14 个百分点，是相对参数量性能最弱的任务。实验发现三类典型问题：

1. **答案错误/不完整**：对多项列举类问题易发生错误或遗漏部分答案。如”Which two platforms are good for B2B companies?”，标准答案”linkedin, facebook”，模型输出”[’facebook’, ’instagram’]”，遗漏 linkedin 并错误添加 instagram；类似地，”Which social media platforms require a low resource and expertise level?” 标准答案为”twitter, facebook”，模型仅输出”[’facebook’]”。
2. **理解偏差**：对于”For which type of businesses is Instagram suitable?”，标准答案为”bakeries & coffee shops, travel agencies, art museums”（具体业务类型），模型输出”[’foot traffic, word-of-mouth awareness, engagement’]”（营销特性描述）。
3. **格式与单位问题**：百分比类问题（如”What percentage are not Americans?”，标准答案”87%”）模型输出裸数字”87”或列表格式”[87]”，在评测中，LLM Judge 通常能识别语义等价，但规范化输出能力仍有待改进。

根本原因在于 InfoVQA 的信息图表包含多层嵌套视觉元素，需同时进行视觉定位、OCR 和推理等。

3.5 MOSS-VL-0708 逐任务综合分析

图 3.4 以折线图对比 MOSS-VL-0708 与各组代表模型在所有任务上的得分走势。

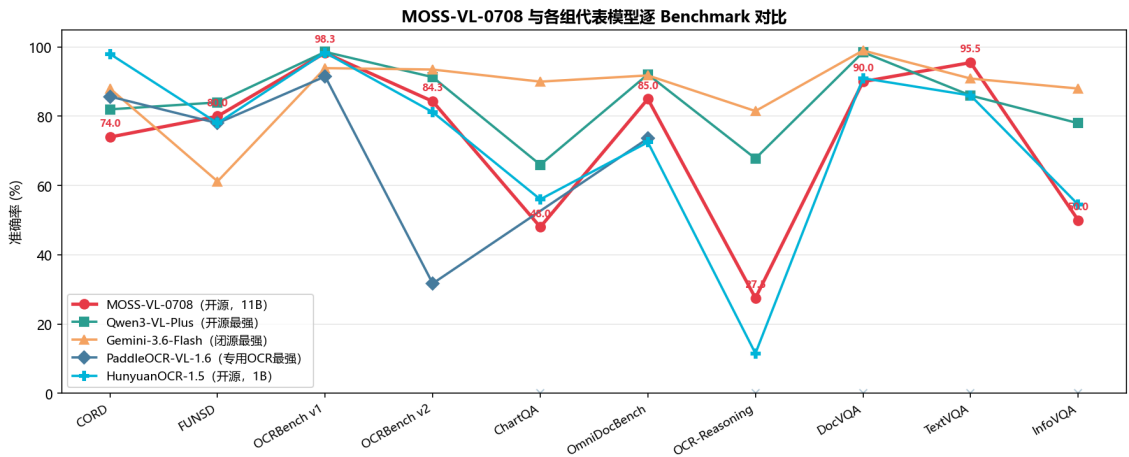


图 3.4 MOSS-VL-0708 与各组代表模型逐 Benchmark 得分对比

- 综合各任务表现，MOSS-VL-0708 呈现出明显的”强 OCR、弱推理”特征：
- **优势任务：**TextVQA（第 1/12，95.5%）、OCRBench v1（第 3/15）、FUNSD（第 3/15），在场景文字理解、纯 OCR 和 KIE 上竞争力强。
 - **中等水平：**OCRBench v2（第 9/15，84.3%）与 DocVQA（第 7/12，90.0%），明显优于多数专用 OCR 模型，但与顶级模型仍有差距。
 - **短板任务：**OCR-Reasoning（27.5%，第 9/11）、ChartQA（48.0%，第 9/12）、InfoVQA（50.0%，第 9/12），在需要多步推理或复杂信息图表理解的任务上显著落后。

表 3.8 汇总了 MOSS-VL-0708 在各任务上的排名及与最强模型的差距。

表 3.8 MOSS-VL-0708 各任务表现汇总

任务	得分	排名 (/参评数)	与最强差距
OCRBench v1	98.3%	3/15	-0.3pp
FUNSD	80.0%	3/15	-4.0pp
TextVQA	95.5%	1/12	+4.6pp
OCRBench v2	84.3%	9/15	-9.2pp
OmniDocBench	85.0%	11/15	-10.5pp
DocVQA	90.0%	7/12	-9.0pp
CORD	74.0%	8/15	-24.0pp
InfoVQA	50.0%	9/12	-39.5pp
ChartQA	48.0%	9/12	-42.0pp
OCR-Reasoning	27.5%	9/11	-54.0pp

3.6 Qwen3-VL 规模扩展分析

图 3.5 分析了 Qwen3-VL 系列不同尺寸在各任务上的性能差异，以观察模型规模对不同任务的影响。

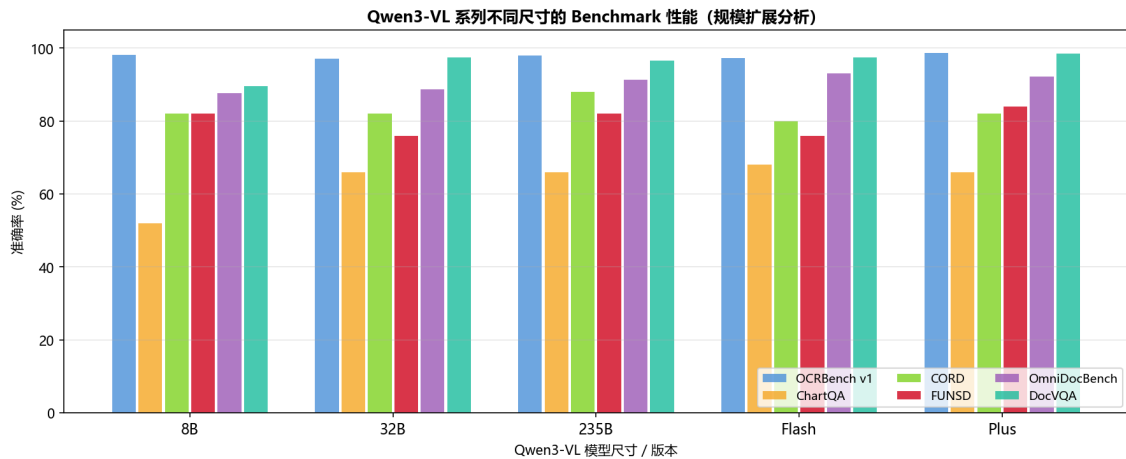


图 3.5 Qwen3-VL 系列规模扩展分析（不同尺寸在各 Benchmark 上的性能）

主要发现如下：

- **OCRBench v1:** 8B (98.2%) $\approx$ 235B (97.9%) $\approx$ Plus (98.6%)，规模对纯文字识别影响极小。
- **ChartQA:** 8B (52.0%) 明显弱于 32B/235B (均 66.0%)，图表推理任务对模型规模更敏感。

- **CORD:** 235B (88.0%) 明显强于 8B/32B (均 82.0%), 键值提取受益于更大模型。
- **FUNSD:** 32B (76.0%) 略弱于 8B/235B (均 82.0%), 规模扩展并非单调递增。

该结果表明, 任务类型是决定模型规模收益的关键因素: 推理密集型任务 (ChartQA、OCR-Reasoning) 更依赖大规模模型, 而文字识别类任务在较小规模下即可达到高水平。

第 4 章 评测结论

4.1 主要结论

本报告对 MOSS-VL-0708 及 14 个对比模型在 10 个权威 Benchmark 上进行了系统性评测。综合实验结果，MOSS-VL-0708 在纯 OCR 文字识别和自然场景文字理解领域具备较强竞争力。在 OCRBench v1 上，MOSS-VL-0708 以 98.3% 位列第三，仅次于 Qwen3-VL-Plus (98.6%) 与 HunyuanOCR-1.5 (98.4%)，超越所有参评闭源大模型（最高 96.4%）和专用 OCR 工具（最高 91.5%）。在 TextVQA 自然场景文字问答任务上，MOSS-VL-0708 以 95.5% 位列第一，超越 Gemini-3.6-Flash (90.9%)、Claude-Sonnet-5 (86.5%) 等所有顶级闭源大模型，证明了开源 VLM 在特定任务上可以达到甚至超越闭源大模型的水平。

在表单 KIE 任务上，MOSS-VL-0708 表现同样突出。FUNSD 表单关键信息提取任务上，MOSS-VL-0708 以 80.0% 位列第 3，超越 GPT-5-Sol (63.2%)、Gemini-3.6-Flash (61.2%)、Claude-Sonnet-5 (59.0%) 等全部闭源大模型。

然而，MOSS-VL-0708 在复杂推理与部分文档 VQA 任务上存在明显短板。InfoVQA (50.0%)、OCR-Reasoning (27.5%)、ChartQA (48.0%) 三项任务与最强模型差距均超过 40 个百分点，是当前版本最薄弱的能力维度。这类任务需要模型在识别文本的基础上，进行多步推理和语义理解，与单纯的文字识别能力要求差异显著。值得注意的是，DocVQA 任务在采用 LLM Judge 评分后得分为 90.0%（排名第 7），远高于之前使用 ANLS 指标时的 37.5%，说明模型在语义理解上具备一定能力，但输出格式与传统评测指标不匹配。同时，OCRBench v2 (84.3%) 虽然明显优于专用 OCR 工具，但与顶级闭源大模型（Gemini 93.5%）仍有 9.2 个百分点差距。

横向对比来看，Qwen3-VL 开源尺寸 (8B/32B/235B) 在 OCRBench、FUNSD 等任务上表现稳定；HunyuanOCR-1.5 在 CORD (98.0%) 与 OCRBench v1 (98.4%) 上表现突出。闭源模型中 Gemini-3.6-Flash 综合最强，在推理与 VQA 类任务（ChartQA 90.0%、DocVQA 99.0%、OCR-Reasoning 81.5%）上全面领先，是当前综合能力最强的参评模型。

另外，Qwen3-VL 规模扩展分析表明，规模扩展对不同任务的收益并不均等。文字识别任务在 8B 规模即可达到饱和，而图表推理任务（ChartQA）从 8B 升至 32B 有明显提升 (52% → 66%)。这一发现对 MOSS-VL-0708 的后续训练策略具有参考意义。

4.2 MOSS-VL-0708 改进方向

基于评测结果，提出以下优先改进方向。

4.2.1 多步推理能力

OCR-Reasoning (27.5%) 的低分根本原因在于模型缺乏足够的多步推理训练数据。可以考虑在训练数据中增加包含完整思维链 (Chain-of-Thought) 的图表数值计算样本，引入强化学习对推理过程进行对齐，奖励包含正确推理步骤的输出，并借鉴 Qwen3-VL 的 CoT 训练范式，使模型能在简短 prompt 下自动展开推理。

4.2.2 信息图表理解能力

ChartQA (48.0%) 中的低分说明模型对图表视觉元素 (颜色、图例、坐标轴) 的理解能力有限。可以增加多样化图表类型 (折线图、饼图、散点图等) 的问答训练数据，引入图表数值精确读取的监督信号，提升对坐标轴刻度的精确感知，并增加跨图表比较推理数据，训练模型处理涉及图表的复合问题。

InfoVQA (50.0%) 上的低分表现了模型在复杂信息图表理解上的缺陷。典型错误包括：**列举不完整** (问 "Which two platforms are good for B2B?" 答 "facebook, instagram" 而非正确的 "linkedin, facebook")、**理解偏差** (问业务类型答营销特性)、**格式问题** (输出列表格式 "['87']" 而非 "87%")。改进方向为增加信息图表训练数据，强化多项答案生成能力，规范输出格式 (带单位、符号)，并训练模型识别题目要求的具体维度。

4.2.3 文档定位读取能力

DocVQA 虽然在 LLM Judge 评分下达到 90.0%，但仍然存在两个主要问题：**数值读取偏差** (如问 "What is the 'actual' value per 1000, during the year 1975?" 答 "0.24" 而非正确的 "0.28")，属于图表刻度读取误差) 和 **表格定位错误** (对会议出席次数等表格数据输出错误值)。改进方向为增加文档版面理解训练数据，强化表格单元格精确定位能力，提升对数值的感知精度 (如小数点后两位)。

4.2.4 结构化文档提取能力

CORD 上 74.0% 的得分落后于专用 OCR 工具 (85.7%) 14 个百分点。收据 KIE 要求精确的字段定位和格式化输出，可以增加收据、发票等半结构化文档的 KIE 训练样本，并引入结构化输出约束 (JSON schema) 确保输出格式的规范性。

4.3 模型选型建议

基于本次评测结果，针对不同应用场景给出选型建议，如表 4.1 所示。

表 4.1 面向不同场景的模型选型建议

应用场景	推荐模型	推荐理由
纯文字识别	Qwen3-VL-Plus / HunyuanOCR-1.5	OCRBench v1 前二
场景文字理解	MOSS-VL-0708	TextVQA 全场第一
发票/表单批量处理	PaddleOCR-VL-1.6	KIE 性能稳定，成本低
文档整页解析（高精度）	Claude-Sonnet-5	OmniDocBench 第一
文档整页解析（批量）	Qwen3-VL-Flash	稳定无失败，成本适中
文档 OCR（低成本）	GLM-OCR	精度高，API 成本低
文档 VQA	Gemini-3.6-Flash	DocVQA 第一
信息图问答	Claude-Sonnet-5	InfoVQA 第一
图表问答与推理	Gemini-3.6-Flash	ChartQA、OCR-Reasoning 双项第一
API 均衡调用	Qwen3-VL-Flash	多任务均衡，调用成本低
开源高精度本地部署	MOSS-VL-0708	TextVQA 第一、OCRBench v1 第三，11B 可本地运行

对于需要 **本地私有化部署**且主要聚焦于 OCR 文字识别和 KIE 的场景，MOSS-VL-0708（11B）是当前最优的开源选择，可在较小的硬件成本下达到接近顶级闭源模型的文字识别效果；若追求极致轻量和速度可选择 HunyuanOCR-1.5。对于需要处理复杂推理或文档 VQA 的场景，建议优先使用 Gemini-3.6-Flash 或 Claude-Sonnet-5。

4.4 评测局限性

本次评测存在以下局限性，在使用评测结论时需注意。首先，各 Benchmark 样本量为 50–250 条，存在一定统计误差，尤其是 CORD、FUNSD、ChartQA 等 50 样本的 Benchmark。其次，使用 qwen3-vl-flash 作为裁判模型可能存在对同系列模型的偏向性，建议后续使用人工评审或多种裁判模型交叉验证。此外，不同模型通过不同渠道调用（专用 OCR 模型走百度智能云 API，其余模型均走代理 API），可能存在版本差异和响应格式差异，难以完全消除。另外，各模型的最优 prompt 格式可能不同，本次统一使用相同 prompt，可能对部分模型存在不利影响。

此外，当前 Benchmark 以英文为主，还可引入中文文档理解（如 CDLA、DuReader-vis）等数据集，评估中文场景下的模型能力差异。此外，也应将推理速度、显存占用、API 成本纳入评测维度，为实际部署提供更全面的决策依据。

参考文献

- [1] Liu H, Li C, Wu Q, et al. LLaVA: Visual instruction tuning[C]//Advances in Neural Information Processing Systems: Vol. 36. 2023.
- [2] Bai J, Bai S, Yang S, et al. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond[A]. 2023.
- [3] Liu H, Chen T, Liu Y, et al. OCRBench: On the hidden mystery of OCR in large multimodal models[A]. 2024.
- [4] Ouyang L, Qu Y, Zhou H, et al. OmniDocBench: Benchmarking diverse PDF document parsing with comprehensive annotations[A]. 2024.
- [5] Bai S, Cai Y, Chen R, et al. Qwen3-VL technical report[A]. 2025.
- [6] Park S, Shin S, Lee B, et al. CORD: A consolidated receipt dataset for post-OCR parsing[C]//Workshop on Document Intelligence at NeurIPS 2019. 2019.
- [7] Jaume G, Ekenel H K, Thiran J P. FUNSD: A dataset for form understanding in noisy scanned documents[C]//2019 International Conference on Document Analysis and Recognition Workshops. 2019: 1-6.
- [8] Fu L, Yang B, Kuang Z, et al. OCRBench v2: An improved benchmark for evaluating large multimodal models on visual text localization and reasoning[A]. 2025.
- [9] Masry A, Long D X, Tan J Q, et al. ChartQA: A benchmark for question answering about charts with visual and logical reasoning[C]//Findings of the Association for Computational Linguistics: ACL 2022. 2022: 2263-2279.
- [10] Huang M, Shi Y, Peng D, et al. OCR-Reasoning benchmark: Unveiling the true capabilities of MLLMs in complex text-rich image reasoning[A]. 2025.
- [11] Mathew M, Karatzas D, Jawahar C. DocVQA: A dataset for VQA on document images[C]//Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision. 2021: 2200-2209.
- [12] Singh A, Natarajan V, Shah M, et al. Towards VQA models that can read[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019: 8317-8326.
- [13] Mathew M, Bagal V, Tito R, et al. InfographicVQA[C]//Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision. 2022: 1697-1706.

附录 A 补充材料

A.1 完整评测数值表

附表 A.1 为完整的 15 模型 $\times$ 10 Benchmark 原始数值表，“—”表示未测试。

表 A.1 完整评测原始数值 (%)

模型	CORD	FUNSD	OCRv1	OCRv2	ChartQA	OmniDoc	OCR-R	DocVQA	TextVQA	InfoVQA
MOSS-VL-0708	74.0	80.0	98.3	84.3	48.0	85.0	27.5	90.0	95.5	50.0
PaddleOCR-VL-1.6	85.7	78.0	91.5	31.7	—	73.6	—	—	—	—
PP-OCRv5	85.7	78.0	91.5	30.9	—	73.9	—	—	—	—
GLM-OCR	56.0	48.0	45.8	38.7	—	94.4	—	—	—	—
DeepSeek-OCR-2	72.0	54.0	76.3	52.2	32.0	70.7	—	51.0	58.0	20.3
Qwen3-VL-Plus	82.0	84.0	98.6	91.3	66.0	92.1	67.8	98.5	86.0	78.0
Qwen3-VL-235B	88.0	82.0	97.9	92.6	66.0	91.3	67.0	96.5	80.0	73.5
Qwen3-VL-Flash	80.0	76.0	97.3	88.7	68.0	93.0	67.3	97.5	47.0	69.5
Qwen3-VL-8B	82.0	82.0	98.2	87.4	52.0	87.6	48.5	89.5	60.0	64.0
Qwen3-VL-32B	82.0	76.0	97.1	92.2	66.0	88.7	64.5	97.5	69.0	77.0
GPT-5-Sol	84.0	63.2	96.1	92.6	82.0	89.3	66.8	98.0	79.5	88.0
Claude-Sonnet-5	80.0	59.0	96.4	91.7	78.0	95.5	74.3	97.5	86.5	89.5
Gemini-3.6-Flash	88.0	61.2	93.9	93.5	90.0	91.8	81.5	99.0	90.9	88.0
Qwen-VL-OCR	64.0	50.0	56.8	75.5	54.0	86.9	8.6	90.0	12.5	45.8
HunyuanOCR-1.5	98.0	78.0	98.4	81.2	56.0	72.5	11.5	91.0	86.0	54.5

注：OCR-R 表示 OCR-Reasoning；OCRv1/v2 表示 OCRBench v1/v2；OmniDoc 表示 OmniDocBench。

MOSS-VL OCR-Reasoning 为 CoT 引导后重测分数（原始无 CoT 为 9.0%）。

Claude-Sonnet-5 / OmniDocBench 仅 156/200 条成功。

A.2 各任务排名汇总

附表 A.2 列出各 Benchmark 的前三名。

表 A.2 各 Benchmark 前三名汇总

Benchmark	第 1	第 2	第 3	MOSS-VL 排名
OCRBench v1	Qwen3-VL-Plus (98.6%)	HunyuanOCR-1.5 (98.4%)	MOSS-VL-0708 (98.3%)	3
OCRBench v2	Gemini-3.6-Flash (93.5%)	Qwen3-VL-235B/GPT-5 (92.6%)	Qwen3-VL-32B (92.2%)	9
FUNSD	Qwen3-VL-Plus (84.0%)	Qwen3-VL-8B/235B (82%)	MOSS-VL-0708 (80.0%)	3
CORD	HunyuanOCR-1.5 (98.0%)	Qwen3-VL-235B/Gemini (88%)	PaddleOCR (85.7%)	8
OmniDocBench	Claude-Sonnet-5 (95.5%)	GLM-OCR (94.4%)	Qwen3-VL-Flash (93%)	11
ChartQA	Gemini-3.6-Flash (90%)	GPT-5-Sol (82%)	Claude-Sonnet-5 (78%)	9
OCR-Reasoning	Gemini-3.6-Flash (81.5%)	Claude-Sonnet-5 (74.3%)	Qwen3-VL-Plus (67.8%)	9
DocVQA	Gemini-3.6-Flash (99%)	Qwen3-VL-Plus (98.5%)	GPT-5-Sol (98%)	7
TextVQA	MOSS-VL-0708 (95.5%)	Gemini-3.6-Flash (90.9%)	Claude/Qwen3-Plus/Hunyuan (86%)	1
InfoVQA	Claude-Sonnet-5 (89.5%)	GPT-5/Gemini (88%)	Qwen3-VL-Plus (78%)	9

A.3 评测框架说明

本次评测使用统一的 Python 评测框架，核心代码结构如下：

- **eval/run_benchmark.py**: 主评测脚本，统一调度模型调用、prompt 构建、结果存储
- **eval/judges/llm_judge.py**: LLM-as-Judge 评分模块，调用 qwen3-vl-flash
- **eval/metrics/ned.py**: NED 字符串相似度评分模块
- **results/scores/**: 原始得分 CSV 文件
- **results/scores/llm_judge/**: LLM Judge 判分 JSON 文件

A.4 实验环境配置

表 A.3 评测实验环境

配置项	说明
操作系统	Ubuntu 22.04 LTS
Python 版本	3.10
评测框架	eval/run_benchmark.py
裁判模型	qwen3-vl-flash (LLM Judge)
评测完成时间	2026-07-27

A.5 评测 Prompt 设计

A.5.1 文字识别类任务 Prompt

以下 Prompt 用于 OCRBench v1/v2、OmniDocBench 等文字识别和文档解析任务，存储于 `configs/prompts.yaml`。

OCR 基础文字识别（`text_recognition`）

- 1 请逐行输出图片中的全部文字，不要添加任何解释。

文档整页解析（`document_parse`）

- 1 请完整转录图片中的所有文字，严格保留原始阅读顺序与版式。
- 2 表格用 `HTML` 表示，公式用 `LaTeX` 表示，其余文本用 `Markdown`。
- 3 不要遗漏、不要臆造。

长文档多页解析（`long_doc`）

- 1 以下是一份多页文档的所有页面图像，按页码顺序排列。请完整
- 2 转录全部内容，严格保留跨页的阅读顺序与版式。表格用 `HTML`
- 3 表示，公式用 `LaTeX` 表示，其余文本用 `Markdown`。请在每页
- 4 内容开头标注页码，如「--- 第1页 ---」。

A.5.2 KIE 结构化信息提取 Prompt

KIE 任务（FUNSD、CORD）采用 `few-shot prompt`，直接嵌入各数据集的 `metadata.jsonl`。

FUNSD 表单关键信息提取

- 1 请识别并输出图片（表单/文件扫描件）中的所有文字，严格按照
- 2 原始阅读顺序逐词输出，不要添加任何格式标记或解释。
- 3
- 4 示例1（表单头部）：
- 5 `TO: John Smith DATE: 01/15/97 FROM: Mary Johnson`
- 6 `SUBJECT: Q4 Budget Review`
- 7
- 8 示例2（传真封面）：
- 9 `TO: K.A. Sparrow FROM: T.D. Bell DATE: 09/17/97 10:55`
- 10 `FAX: 603 841 1898 LORILLARD PTLD 001`
- 11
- 12 注意：
- 13 - 直接输出识别到的文字，保持原始词序
- 14 - 不要换行，不要添加 `Markdown` 格式
- 15 - 如果某处文字模糊，尽力识别并输出

CORD 收据键值提取

```
1 请从收据图片中提取结构化信息，严格按照以下JSON格式输出，
2 不要添加任何解释：
3
4 示例1（单商品收据）：
5 {
6   "menu": [{"nm": "COFFEE LATTE", "cnt": "1", "price":
7     "4.500"}],
8   "sub_total": {"subtotal_price": "4.500", "tax_price":
9     "0.450"},
10  "total": {"total_price": "4.500", "cashprice": "5.000",
11    "changeprice": "0.500"}
12 }
13
14 示例2（多商品收据）：
15 {
16   "menu": [
17     {"nm": "BURGER SET", "cnt": "2", "price": "15.000"},
18     {"nm": "COLA", "cnt": "2", "price": "3.000"}
19   ],
20   "total": {"total_price": "17.000", "creditcardprice":
21     "17.000"}
22 }
```

21 字段说明：menu.nm=商品名称，menu.cnt=数量，
22 total.total_price=合计金额（必填）

A.5.3 VQA 和推理任务 Prompt

VQA 类任务（DocVQA、TextVQA、InfoVQA）直接使用各数据集官方提供的问题（prompt 字段），不附加额外指令。

OCR-Reasoning CoT 引导 Prompt OCR-Reasoning 任务中，原始数据集只提供简短问题（如“What is the proportion of...”），若直接提问，模型倾向于输出简短数值而非完整推理过程。为与 GT 的 step-by-step 格式对齐，本评测在原问题外附加 CoT 引导（实现于 eval/run_benchmark.py）：

```
1 Please solve this problem step by step.
2 Show your reasoning process clearly.
3
4 Question: {原始问题}
5
6 Instructions:
7 1. First, carefully examine the chart/image and
8   identify all relevant data
9 2. Break down your solution into clear steps
10  (step1, step2, etc.)
11 3. Show all calculations and reasoning
```

```
12 4. State your final answer clearly
13
14 Your detailed solution:
```

该 Prompt 使 MOSS-VL-0708 的 OCR-Reasoning 准确率从原始的 9.0%（无 CoT）提升至 27.5%（有 CoT），提升幅度约为 3 倍。

A.6 LLM Judge Prompt 设计

本评测采用 qwen3-vl-flash 作为裁判模型（LLM-as-Judge），针对不同任务使用定制化 prompt。所有 judge 调用均设置 temperature=0, max_tokens=10，只返回 CORRECT 或 WRONG 一行。

A.6.1 通用 VQA Judge Prompt

用于 DocVQA、TextVQA、InfoVQA、ChartQA 等问答任务：

```
1 你是一个严格的 VQA 评测裁判。
2
3 给你一道视觉问答题的标准答案（GT）和模型的回答（Pred），
4 判断模型回答是否正确。
5
6 判断规则：
7 - 答案语义等价即为正确，忽略大小写、标点、单复数差异
8 - 数字答案：允许合理格式差异
9   ("3.5 million" vs "3,500,000" 视为正确)
10 - 若 GT 为多个参考答案之一，Pred 与任意一个匹配即为
11   CORRECT
12 - 若 Pred 包含额外无关内容但核心答案正确，判为 CORRECT
13 - 若 Pred 明显答错，判为 WRONG
14
15 只回复一行：CORRECT 或 WRONG
16
17 GT: {gt}
18 Pred: {pred}
```

A.6.2 OCR-Reasoning Judge Prompt

用于 OCR-Reasoning 等需要多步推理的任务，强调只判断最终答案的正确性：

```
1 你是一个严格的评测裁判，负责判断模型对图表推理题的回答
2 是否正确。
3
4 标准答案（GT）包含完整的推理步骤和最终数值/结论。
5 模型预测（Pred）也可能包含推理过程，你只需关注最终答案
6 是否正确。
7
8 判断规则：
```

```

9 - 只比较最终答案的正确性，忽略推理过程的差异
10 - 数值答案：精确匹配或在合理精度范围内
11   （如 3.625 vs 3.6 可接受）
12 - 实体/类别答案：语义等价即可
13   （如 "China" vs "中国"）
14 - 如果模型给出了错误的最终数字或错误的实体，判为 WRONG
15
16 只回复一行：CORRECT 或 WRONG
17
18 GT（含推理）：
19 {gt}
20
21 模型预测：
22 {pred}

```

A.6.3 OmniDocBench Judge Prompt

用于文档整页解析任务，评估内容完整性，对格式差异宽容：

```

1 你是一个严格的文档解析评测裁判。
2
3 给你一段文档页面的标准内容（GT）和模型的解析输出（Pred），
4 判断模型解析是否基本正确。
5
6 判断规则：
7 - 主要内容（标题、正文关键文字、数值、表格核心数据）
8   基本覆盖：CORRECT
9 - 公式：LaTeX 表达式语义等价即可，不要求格式完全相同
10 - 表格：关键单元格内容正确即可，
11   不要求 HTML 格式完全一致
12 - 允许轻微的文字顺序差异、标点差异
13 - 如果大量关键内容缺失或错误，判为 WRONG
14 - GT 较短时（<100字），要求更严格，
15   主要内容必须全部出现
16
17 只回复一行：CORRECT 或 WRONG
18
19 GT（标准内容，前800字）：
20 {gt}
21
22 模型预测（前800字）：
23 {pred}

```

A.7 核心代码

A.7.1 模型适配器基类（adapters/base.py）

所有模型适配器继承自统一基类，规范了 `infer`（单图）和 `infer_multi_image`（多图）接口。

Listing A.1 模型适配器基类

```

1 from abc import ABC, abstractmethod
2
3 class ModelAdapter(ABC):
4     """所有模型推理适配器的基类。"""
5     model_key: str = "" # configs/models.yaml 中的键名
6
7     @abstractmethod
8     def infer(self, image_path: str, prompt: str) -> str:
9         """单图推理，返回模型输出的纯文本。"""
10
11     def infer_multi_image(
12         self, image_paths: list[str], prompt: str) -> str:
13         """多图推理（用于长文档评测）。
14         默认实现：逐页单独推理后按页码顺序拼接。
15         支持多图输入的子类应覆盖此方法。
16         """
17         parts = []
18         for i, path in enumerate(image_paths, start=1):
19             page_prompt = (
20                 f"这是第 {i} 页，共 {len(image_paths)} 页。\\n"
21                 + prompt
22             )
23             result = self.infer(path, page_prompt)
24             parts.append(f"--- 第{i}页 ---\\n{result}")
25         return "\\n\\n".join(parts)

```

A.7.2 通用 OpenAI 兼容适配器（adapters/api_models.py）

所有闭源 API 模型（GPT-5.6-Sol、Gemini-3.6-Flash、Claude-Sonnet-5）及 MOSS-VL 的云端服务均通过此适配器调用。

Listing A.2 OpenAI 兼容适配器（含图片自动压缩）

```

1 def _encode_image(
2     image_path: str, max_size_mb: float = 5.0
3 ) -> tuple[str, str]:
4     """返回 (base64_data, mime_type)。超过限制自动压缩。"""
5     path = Path(image_path)
6     mime = {".jpg": "image/jpeg", ".jpeg": "image/jpeg",
7             ".png": "image/png"}.get(path.suffix.lower(), "image
8             /jpeg")
9
10    if path.stat().st_size / 1024 / 1024 <= max_size_mb:
11        with open(path, "rb") as f:
12            return base64.b64encode(f.read()).decode(), mime
13
14    img = Image.open(path).convert("RGB")
15    w, h = img.size
16    if max(w, h) > 2048: # 先缩分辨率
17        scale = 2048 / max(w, h)

```

```

17         img = img.resize((int(w * scale), int(h * scale)),
18                           Image.LANCZOS)
19     quality = 85
20     while quality >= 20:                # 再逐步降质量
21         buf = io.BytesIO()
22         img.save(buf, format="JPEG", quality=quality)
23         if buf.tell() / 1024 / 1024 <= max_size_mb:
24             break
25         quality -= 10
26     buf.seek(0)
27     return base64.b64encode(buf.read()).decode(), "image/jpeg"
28
29
30 class OpenAICompatAdapter(ModelAdapter):
31     """通用 OpenAI 兼容接口适配器（支持所有模型）。"""
32     def __init__(self, model_id: str, api_base: str,
33                 api_key: str, max_tokens: int = 4096):
34         from openai import OpenAI
35         self.model_id = model_id
36         self.max_tokens = max_tokens
37         self.client = OpenAI(api_key=api_key, base_url=api_base)
38
39     def infer(self, image_path: str, prompt: str) -> str:
40         return _retry_with_backoff(
41             self._call, [image_path], prompt)
42
43     def _call(self, image_paths: list[str], prompt: str) -> str:
44         content = []
45         for p in image_paths:
46             b64, mime = _encode_image(p)
47             content.append({
48                 "type": "image_url",
49                 "image_url": {"url": f"data:{mime};base64,{b64}"
50                                 },
51             })
52         content.append({"type": "text", "text": prompt})
53         resp = self.client.chat.completions.create(
54             model=self.model_id,
55             messages=[{"role": "user", "content": content}],
56             temperature=0,
57             max_tokens=self.max_tokens,
58         )
59         return resp.choices[0].message.content.strip()

```

A.7.3 评测主流程（eval/run_benchmark.py）核心代码

评测脚本支持断点续跑（resume），对每个样本调用适配器推理并逐行写入 JSONL 文件。

Listing A.3 评测主流程（断点续跑 + 逐样本落盘）

```

1 def run_benchmark(model_key: str, benchmark: str,

```

```

2         data_dir: str, out_dir: str,
3         task: str = "document_parse",
4         resume: bool = True,
5         max_samples: int = 0) -> Path:
6     data_dir = Path(data_dir)
7     out_file = Path(out_dir) / model_key / f"{benchmark}.jsonl"
8     out_file.parent.mkdir(parents=True, exist_ok=True)
9
10    prompts = _load_prompts(config_dir)
11    samples = _load_dataset(data_dir, task)
12
13    # 断点续跑: 跳过已完成的样本 ID
14    done_ids: set[str] = set()
15    if resume and out_file.exists():
16        with open(out_file) as f:
17            for line in f:
18                done_ids.add(json.loads(line)["id"])
19
20    adapter = load_adapter(model_key)
21    pending = [s for s in samples if s["id"] not in done_ids]
22    if max_samples > 0:
23        pending = pending[:max(0, max_samples - len(done_ids))]
24
25    with open(out_file, "a", encoding="utf-8") as fout:
26        for i, sample in enumerate(pending):
27            prompt = _get_prompt(sample, task, prompts)
28            image_paths = _get_image_paths(sample, data_dir)
29            t0 = time.perf_counter()
30            try:
31                if len(image_paths) > 1:
32                    output = adapter.infer_multi_image(
33                        image_paths, prompt)
34                else:
35                    output = adapter.infer(image_paths[0],
36                                           prompt)
37                status, error = "ok", ""
38            except Exception as e:
39                output, status, error = "", "error", str(e)
40            record = {
41                "id": sample["id"], "model": model_key,
42                "benchmark": benchmark, "task": task,
43                "pred": output, "gt": sample.get("gt", ""),
44                "elapsed": round(time.perf_counter() - t0, 2),
45                "status": status, "error": error,
46            }
47            fout.write(json.dumps(record, ensure_ascii=False) +
48                        "\n")
49            fout.flush() # 每条立即落盘, 保证断点可恢复

```

A.7.4 并行 LLM Judge 评分 (eval/llm_judge_parallel.py, 核心部分)

采用 ThreadPoolExecutor 多线程并行调用裁判模型，支持断点续跑和实时进度显示。

Listing A.4 并行 LLM Judge 评分 (16 workers)

```
1 def judge_model(model_key: str, benchmark: str,
2                 raw_dir: Path, valid_ids: set,
3                 out_dir: Path, workers: int = 16):
4     raw_file = raw_dir / model_key / f"{benchmark}.jsonl"
5     records = {}
6     with open(raw_file) as f:
7         for line in f:
8             r = json.loads(line)
9             rid = r["id"]
10            if rid in valid_ids and rid not in records \
11                and r["status"] == "ok":
12                records[rid] = r
13
14    # 断点续跑：读取已有评分结果
15    out_file = out_dir / f"{model_key}_{benchmark}.json"
16    done_results = {}
17    if out_file.exists():
18        existing = json.load(open(out_file))
19        done_results = {r["id"]: r
20                        for r in existing.get("details", [])}
21
22    pending = {rid: r for rid, r in records.items()
23               if rid not in done_results}
24    client = load_client()
25
26    def judge_task(item):
27        rid, r = item
28        # 支持 gt_list (多个正确答案)
29        gt_list = r.get("gt_list", [r["gt"]]) or [r["gt"]]
30        correct = any(
31            judge_one(client, gt, r["pred"], benchmark)
32            for gt in gt_list
33        )
34        return {"id": rid, "correct": correct,
35               "gt": r["gt"][:200], "pred": r["pred"][:200]}
36
37    new_results = dict(done_results)
38    with ThreadPoolExecutor(max_workers=workers) as ex:
39        futs = {ex.submit(judge_task, item): item[0]
40                 for item in pending.items()}
41        for fut in as_completed(futs):
42            detail = fut.result()
43            new_results[detail["id"]] = detail
44
45    all_details = list(new_results.values())
46    correct = sum(1 for d in all_details if d["correct"])
```

```

47     result = {
48         "model": model_key, "benchmark": benchmark,
49         "accuracy": round(correct / len(all_details), 4),
50         "correct": correct, "total": len(all_details),
51         "details": all_details,
52     }
53     json.dump(result, open(out_file, "w"), ensure_ascii=False)

```

A.7.5 NED 字符准确率评分 (eval/metrics/ned.py)

OCRBench v1 使用归一化编辑距离 (NED) 计算字符准确率, 不区分大小写。

- **eval/run_benchmark.py**: 主评测脚本, 统一调度模型调用、prompt 构建、结果存储与断点续跑
- **eval/llm_judge_parallel.py**: 并行 LLM-as-Judge 评分模块 (16 workers), 调用 qwen3-vl-flash
- **eval/metrics/ned.py**: OCRBench v1 字符串相似度 (NED) 评分模块
- **configs/prompts.yaml**: 文字识别类任务统一 Prompt 配置
- **results/scores/**: 原始得分 CSV 文件
- **results/scores/llm_judge/**: LLM Judge 逐样本判分 JSON 文件